

**AWS
PROJECTS
HANDBOOK**

VERIQTA
ENGINEERING EXCELLENCE

**20 HANDS-ON
PROJECTS
FOR BEGINNERS**



AWS BEGINNER-FRIENDLY PROJECTS



H A N D B O O K

BUILD REAL-WORLD CLOUD PROJECTS. LEARN BY DOING.



EC2



S3



RDS



ALB



LAMBDA



CLOUDWATCH



IAM



DYNAMODB



VPC



API GATEWAY



- ✓ Step-by-Step Guides
- ✓ Beginner Friendly
- ✓ Real-World Scenarios
- ✓ Best Practices
- ✓ Hands-On Learning
- ✓ Job-Ready Skills

★★★
**20
PRACTICAL
PROJECTS**
★★★

SMALL PROJECTS. BIG SKILLS. REAL CLOUD EXPERIENCE.



**BUILD
CONFIDENCE**



**STRENGTHEN
YOUR PORTFOLIO**



**PREPARE FOR
CLOUD JOBS**



**THINK CLOUD
ACT CLOUD**

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



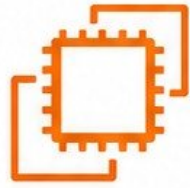
X
veriqta



Instagram
veriqta



Telegram
veriqta



1. LAUNCH YOUR FIRST EC2 WEB SERVER



BUILD. CONNECT. DEPLOY. TEST.

1 CREATE AN EC2 INSTANCE



- Go to EC2 Dashboard
- Click "Launch Instance"
- Give your instance a name
- Review and click "Launch Instance"



TIP: This is your cloud computer.

2 CHOOSE AN AMI AND INSTANCE TYPE



- Choose Amazon Linux 2023 (AMI)
- Select instance type: t2.micro (Free Tier eligible)
- Review details



TIP: t2.micro is perfect for learning.

3 CONFIGURE A SECURITY GROUP

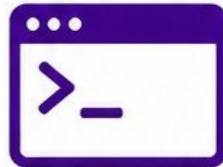


- Create a new security group
- Allow SSH (Port 22) – Your IP
- Allow HTTP (Port 80) – 0.0.0.0/0
- Allow HTTPS (Port 443) – 0.0.0.0/0
- Review and launch



TIP: These rules allow traffic to your server.

4 CONNECT TO THE INSTANCE



- Download your .pem key file
- Change permissions: `chmod 400 your-key.pem`
- Connect using SSH: `ssh -i your-key.pem ec2-user@your-public-ip`
- You are now connected!



TIP: Keep your .pem key file safe.

5 INSTALL A WEB SERVER



- Update the system: `sudo yum update -y`
- Install Apache: `sudo yum install -y httpd`
- Start Apache: `sudo systemctl start httpd`
- Enable Apache on boot: `sudo systemctl enable httpd`



TIP: Apache is the most popular web server.

6 TEST FROM A BROWSER



- Open your browser
- Enter your instance public IP (e.g., `http://YOUR-PUBLIC-IP`)
- You should see the Apache default page!
- Congratulations! Your web server is running.



TIP: Your website is now live on the internet.



WHAT YOU JUST DID

- ✓ Launched an EC2 instance
- ✓ Selected an AMI and instance type
- ✓ Configured a security group
- ✓ Connected to your instance
- ✓ Installed a web server
- ✓ Tested from a browser



NEXT STEP

Great job! You have launched your first web server in AWS. Keep building!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta

VERIQTA SOCIAL



X
veriqta



Instagram
veriqta



Telegram
veriqta



2. HOST A STATIC WEBSITE ON AMAZON S3



SIMPLE. SECURE. SCALABLE.

1 CREATE AN S3 BUCKET



- Go to S3 Console
- Click "Create bucket"
- Enter a unique bucket name
- Choose a Region
- Keep default settings
- Click "Create bucket"



TIP: Bucket names must be globally unique.

2 UPLOAD WEBSITE FILES



- Open your bucket
- Click "Upload"
- Add files: index.html, style.css, script.js, images, etc.
- Click "Upload"



TIP: index.html is the default home page.

3 CONFIGURE STATIC WEBSITE HOSTING



- Go to bucket → Properties
- Scroll to "Static website hosting"
- Select "Enable"
- Set Index document: index.html
- (Optional) Set Error document: error.html
- Click "Save changes"



TIP: Your website will be hosted at the provided endpoint.

4 UNDERSTAND OBJECT PERMISSIONS



- For a public website, allow public read access to objects
- Go to bucket → Permissions → Block Public Access (ensure settings allow public access)
- Bucket Policy example:

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::your-bucket/*"
}
```



TIP: Never store sensitive data in publicly accessible buckets.

5 TEST THE WEBSITE ENDPOINT



- Copy the "Bucket website endpoint" from the Properties tab
- Open the endpoint in your browser
- You should see your website live!
- Test different pages and assets
- If not working, review permissions and file names



TIP: Endpoint looks like:
<http://your-bucket-name.s3-website-region.amazonaws.com>

6 CLEAN UP RESOURCES



- Empty the bucket (remove all objects)
- Delete the bucket
- This prevents unnecessary charges
- Good cloud practice!



TIP: Always clean up after completing a project.



WHAT YOU JUST DID

- ✓ Created an S3 bucket
- ✓ Uploaded website files
- ✓ Enabled static website hosting
- ✓ Configured permissions
- ✓ Tested the website endpoint
- ✓ Cleaned up resources



NEXT STEP

Great job! You have hosted your first static website on Amazon S3. Keep building!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta

VERIQTA SOCIAL



X
veriqta



Instagram
veriqta



Telegram
veriqta



3. CREATE A SECURE IAM USER



SECURE ACCESS. LEAST PRIVILEGE. TOTAL CONTROL.

1 IAM USERS AND GROUPS



- IAM is global (not region-specific)
- Users represent individuals
- Groups help manage permissions easily
- Policies define what actions are allowed or denied



TIP: Use groups to assign permissions, not users directly.

2 CREATE A USER



- Go to IAM Console
- Click "Users" → "Add users"
- Enter a user name (e.g., dev-user)
- Select "Programmatic access" or "AWS Management Console access"
- Review and create the user



TIP: Give users meaningful names that describe their purpose.

3 ASSIGN PERMISSIONS



- Add the user to a group
- Attach a policy to the group
- Use managed policies or create custom policies
- Review permissions



TIP: Grant only the permissions needed to perform the job (Least Privilege).

4 LEAST PRIVILEGE



- Give only the minimum permissions required
- Avoid AdministratorAccess
- Use job-based access
- Review and refine policies regularly



TIP: Less access means more security.

5 MFA (MULTI-FACTOR AUTHENTICATION)



- Enable MFA for the user
- Choose "Virtual MFA device"
- Scan the QR code with an authenticator app
- Enter the 6-digit code to activate
- MFA adds an extra layer of security



TIP: Always enable MFA for console access users.

6 TEST PERMITTED AND DENIED ACTIONS



- Log in as the new user
- Try allowed actions (should work)
- Try restricted actions (should fail)
- Verify access based on the assigned policy
- Document results



TIP: Test regularly to ensure policies work as expected.



WHAT YOU JUST DID

- ✓ Understood IAM users and groups
- ✓ Created a secure IAM user
- ✓ Assigned permissions using groups
- ✓ Applied the principle of least privilege
- ✓ Enabled MFA for the user
- ✓ Tested allowed and denied actions



NEXT STEP

Great job! You have created a secure IAM user with least privilege access. Keep building!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQTA SOCIAL



4. BUILD YOUR FIRST CUSTOM VPC



YOUR OWN NETWORK. YOUR RULES.

1 CREATE A VPC



- Go to VPC Dashboard
- Click "Create VPC"
- Enter a name (e.g., MyVPC)
- Choose IPv4 CIDR block (e.g., 10.0.0.0/16)
- Click "Create VPC"



TIP: A /16 CIDR block gives you 65,536 IP addresses.

2 CREATE PUBLIC AND PRIVATE SUBNETS

PUBLIC SUBNET



PRIVATE SUBNET



- Go to Subnets → "Create subnet"
- Select your VPC
- Create a public subnet (e.g., 10.0.1.0/24)
- Create a private subnet (e.g., 10.0.2.0/24)
- Choose different Availability Zones
- Click "Create subnet"



TIP: Use different AZs for high availability.

3 ROUTE TABLES



- Go to Route Tables → "Create route table"
- Name it (e.g., Public-RT)
- For public subnet: add route 0.0.0.0/0 → IGW
- For private subnet: (keep default local route only)
- Associate public RT with public subnet
- Associate private RT with private subnet



TIP: Route tables control where network traffic goes.

4 INTERNET GATEWAY



- Go to Internet Gateways
- Click "Create internet gateway"
- Name it (e.g., MyIGW)
- Attach it to your VPC
- This allows your public subnet to access the internet



TIP: Only resources in public subnets can access the internet through an IGW.

5 CIDR BASICS



- CIDR defines your IP address range
- /16 = 65,536 IPs (10.0.0.0 – 10.0.255.255)
- /24 = 256 IPs (common for subnets)
- Subnets must not overlap
- Plan your IP ranges carefully



TIP: Good CIDR planning avoids IP conflicts later.

6 VERIFY NETWORK ROUTING



- Launch an EC2 instance in public subnet → Should access the internet
- Launch an EC2 instance in private subnet → Should NOT access the internet
- Use ping, curl, or browser to test
- Check route tables and associations if something fails



TIP: Test early to catch routing issues before deployment.



WHAT YOU JUST DID

- ✓ Created a VPC
- ✓ Created public and private subnets
- ✓ Configured route tables
- ✓ Attached an Internet Gateway
- ✓ Understood CIDR basics
- ✓ Verified network routing



NEXT STEP

Now you can launch resources inside your VPC and build secure, scalable architectures!



VERIQTA SOCIAL



5. DEPLOY EC2 INSIDE YOUR CUSTOM VPC



LAUNCH. CONNECT. VERIFY. TROUBLESHOOT.

1 LAUNCH EC2 INTO A SUBNET



- GO TO EC2 CONSOLE
- CLICK "LAUNCH INSTANCE"
- CHOOSE AN AMI
- CHOOSE INSTANCE TYPE
- UNDER "NETWORK SETTINGS":
 - SELECT YOUR VPC
 - SELECT A SUBNET (PUBLIC OR PRIVATE)
- CLICK "LAUNCH INSTANCE"



TIP: FOR INTERNET ACCESS, USE A PUBLIC SUBNET. FOR PRIVATE, NO PUBLIC IP.

2 PUBLIC IP ADDRESSING



- IN "NETWORK SETTINGS" ENABLE "AUTO-ASSIGN PUBLIC IP" FOR PUBLIC SUBNET
- OR MANUALLY ASSIGN ELASTIC IP AFTER LAUNCH
- PUBLIC IP = REACHABLE FROM THE INTERNET
- PRIVATE IP = INTERNAL ONLY



TIP: PUBLIC SUBNET + IGW + ROUTE = INTERNET ACCESS.

3 SECURITY GROUPS

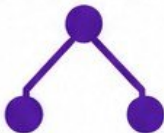


- CHOOSE AN EXISTING SG OR CREATE A NEW ONE
- ALLOW SSH (PORT 22) FROM YOUR IP
- ALLOW HTTP (PORT 80) OR HTTPS (PORT 443) IF NEEDED
- DO NOT ALLOW 0.0.0.0/0 FOR SSH!



TIP: SECURITY GROUPS ACT AS A VIRTUAL FIREWALL FOR YOUR INSTANCE.

4 NETWORK CONNECTIVITY



- PUBLIC SUBNET SHOULD HAVE ROUTE TO INTERNET GATEWAY
- PRIVATE SUBNET SHOULD HAVE ROUTE TO NAT GATEWAY (IF NEEDED)
- CHECK ROUTE TABLES
- ENSURE SG ALLOWS REQUIRED TRAFFIC
- VERIFY NACLs (IF USED)



TIP: CORRECT ROUTES + SG RULES = SUCCESSFUL CONNECTIVITY.

5 SSH ACCESS



- DOWNLOAD YOUR .PEM KEY FILE
- SET PERMISSIONS: `chmod 400 YOUR-KEY.PEM`
- CONNECT USING SSH: `ssh -i YOUR-KEY.PEM EC2-USER@YOUR-PUBLIC-IP`
- YOU SHOULD SEE THE COMMAND PROMPT



TIP: KEEP YOUR .PEM KEY FILE SAFE AND PRIVATE.

6 TROUBLESHOOT FAILED CONNECTIONS



- CHECK YOUR PUBLIC IP (CORRECT?)
- IS INSTANCE RUNNING?
- IS SG ALLOWING SSH FROM YOUR IP?
- IS THE INSTANCE IN THE CORRECT SUBNET?
- IS THE ROUTE TABLE CORRECT?
- CHECK NETWORK ACLs
- USE EC2 "INSTANCE CONNECT" OR SSM SESSION MANAGER AS ALTERNATIVE



TIP: GO STEP-BY-STEP. ONE SMALL ISSUE CAN BLOCK THE CONNECTION.



WHAT YOU JUST DID

- ✓ LAUNCHED EC2 IN YOUR CUSTOM VPC
- ✓ CONFIGURED PUBLIC IP ADDRESSING
- ✓ SET UP SECURITY GROUPS
- ✓ VERIFIED NETWORK CONNECTIVITY
- ✓ CONNECTED USING SSH
- ✓ TROUBLESHOT POTENTIAL ISSUES



NEXT STEP

Install a web server and test from your browser! You're building real infrastructure!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQTA SOCIAL



6. CREATE A SIMPLE DATABASE WITH AMAZON RDS



RELIABLE. SECURE. MANAGED.

1 LAUNCH AN RDS DATABASE

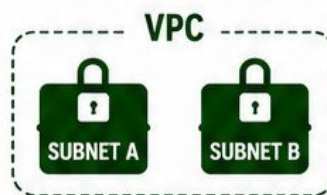


- Go to RDS Console
- Click "Create database"
- Choose engine (e.g., MySQL, PostgreSQL)
- Select a template: Free tier or Dev/Test
- Configure settings
- Click "Create database"



TIP: Use Free tier for practice and learning.

2 DATABASE SUBNET CONCEPTS



- Use a DB Subnet Group
- Select at least two subnets in different Availability Zones
- Subnets should be private
- RDS deploys in your subnets for high availability



TIP: Private subnets keep your database secure.

3 SECURITY GROUPS



- Create or use an existing SG
- Allow inbound access only from specific sources
- Add rule:
 - Type: Database (e.g., MySQL/Aurora)
 - Port: 3306 (MySQL) or 5432 (PostgreSQL)
 - Source: App server IP or SG
- Do NOT allow 0.0.0.0/0



TIP: Restrict access to only trusted IPs or security groups.

4 CONNECT AN APPLICATION/CLIENT



- Get the endpoint from RDS Console
- Use the endpoint, port, username, and password
- Update your application's connection settings
- Example (MySQL):
`jdbc:mysql://your-endpoint:3306/dbname`



TIP: Never hardcode passwords in your application.

5 TEST DATABASE CONNECTIVITY



- Use a MySQL/PostgreSQL client (e.g., DBeaver, MySQL Workbench) or terminal
- Connect using endpoint, port, username, and password
- Run a simple query (e.g., `SELECT 1;`)
- Confirm successful connection



TIP: Test from your app server or allowed client IP.

6 PROTECT DATABASE ACCESS



- Keep your DB in private subnets
- Restrict SG inbound rules
- Enable encryption (at rest and in transit)
- Enable automated backups
- Use strong passwords
- Monitor with CloudWatch
- Regularly review access



TIP: Security today prevents incidents tomorrow.



WHAT YOU JUST DID

- ✓ Launched an RDS database
- ✓ Configured DB subnet group
- ✓ Set up security groups
- ✓ Connected an application/client
- ✓ Tested database connectivity
- ✓ Protected database access



NEXT STEP

Great job! You now have a secure, managed database running on Amazon RDS. Use it in your applications and keep building!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQTA SOCIAL



7. BUILD A TWO-TIER WEB APPLICATION



SEPARATE. SECURE. SCALABLE.

1 EC2 APPLICATION SERVER



- Launch an EC2 instance in a public or private subnet
- Install your web application (e.g., Apache, Nginx, Node.js)
- Ensure the application runs and listens on the correct port
- Allow HTTP/HTTPS from users via Security Group



TIP: Keep your application layer separate from the database layer.

2 RDS DATABASE

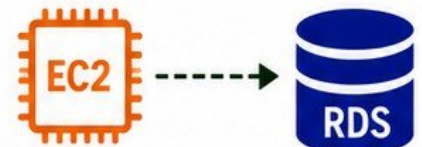


- Launch an RDS database
- Choose engine (e.g., MySQL, PostgreSQL)
- Deploy in private subnets
- Do NOT make the database public
- Note the endpoint, port, username, and password



TIP: RDS should always be in private subnets.

3 APPLICATION-TO-DATABASE CONNECTIVITY



- In your application, configure the database endpoint, port, username, and password
- Ensure the application can connect to the database
- Use connection pooling for better performance



TIP: Use private IP communication inside your VPC.

4 SECURITY-GROUP REFERENCES



- EC2 SG should allow outbound traffic to the RDS SG on the database port (e.g., 3306, 5432)
- RDS SG should allow inbound traffic only from the EC2 SG
- Do NOT allow access from 0.0.0.0/0
- This keeps your database secure



TIP: SG references are more secure than CIDRs.

5 TEST THE ARCHITECTURE



- Access your web application from a browser
- Perform actions that require database interaction
- Verify data is stored in and retrieved from RDS
- Check logs and confirm end-to-end functionality



TIP: Test all major features end-to-end.

6 BASIC TROUBLESHOOTING



- Cannot connect to database? Check SG rules and subnet placement
- Application error? Check app logs and database credentials
- Timeout issues? Check network ACLs and routing
- Database not reachable? Verify endpoint and port



TIP: Logs and SG rules solve 80% of issues.



WHAT YOU JUST DID

- ✓ Launched an EC2 application server
- ✓ Secured communication with SG references
- ✓ Provisioned an RDS database
- ✓ Tested the two-tier application
- ✓ Configured application connectivity
- ✓ Troubleshot basic issues



NEXT STEP

Improve your application by adding auto scaling, backups, monitoring, and load balancing!



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta

VERIQTA SOCIAL



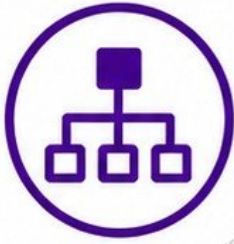
X
veriqta



Instagram
veriqta



Telegram
veriqta

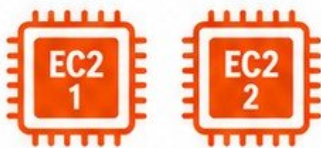


8. CREATE YOUR FIRST APPLICATION LOAD BALANCER



HIGH AVAILABILITY. SCALABLE. RELIABLE.

1 LAUNCH TWO EC2 INSTANCES



- Launch two EC2 instances in the same VPC but in different Availability Zones.
- Install a web server (e.g., Apache or Nginx) on both instances.
- Ensure both instances are running.

TIP: Use a t2.micro or t3.micro for practice.

2 CREATE A TARGET GROUP



- Go to EC2 > Target Groups.
- Click Create target group.
- Choose Instance as target type.
- Select HTTP (port 80) or your application port.
- Register both EC2 instances.

TIP: Target groups route traffic to healthy instances only.

3 CONFIGURE HEALTH CHECKS



HEALTHY

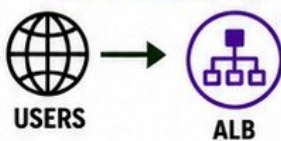


UNHEALTHY

- Set protocol: HTTP
- Set path: / or your health endpoint (e.g., /health)
- Set healthy threshold: 2
- Set unhealthy threshold: 2
- Set interval: 30 seconds
- Set timeout: 5 seconds

TIP: Proper health checks ensure traffic goes only to healthy instances.

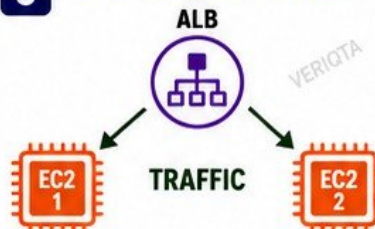
4 CREATE AN APPLICATION LOAD BALANCER



- Go to EC2 > Load Balancers.
- Click Create load balancer.
- Choose Application Load Balancer.
- Select your VPC and public subnets.
- Create a listener (HTTP:80) and forward to the target group.

TIP: ALB must be in public subnets and have an Internet Gateway.

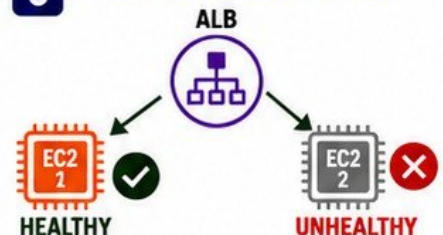
5 DISTRIBUTE TRAFFIC



- The ALB distributes incoming traffic across both healthy instances.
- Traffic is spread automatically based on the load.
- If one instance becomes unhealthy, it stops receiving traffic.

TIP: ALB automatically handles failover and load distribution.

6 TEST INSTANCE FAILURE



- Stop or terminate one EC2 instance.
- Wait for the health check to mark it unhealthy.
- Verify that traffic is now routed only to the healthy instance.
- Start the instance again and confirm it returns to service.

TIP: ALB ensures your application stays available even if an instance fails.

WHAT YOU JUST DID



- ✓ Launched two EC2 instances
- ✓ Created a target group
- ✓ Configured health checks
- ✓ Created an Application Load Balancer
- ✓ Distributed traffic across instances
- ✓ Tested instance failure and failover



NEXT STEP

Secure your ALB with HTTPS, add a domain with Route 53, and scale with Auto Scaling Group.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



9. BUILD AN AUTO SCALING WEB APPLICATION



AUTO SCALE. HIGH AVAILABILITY. COST EFFICIENT.

1 CREATE A LAUNCH TEMPLATE



- Go to EC2 > Launch Templates.
- Click Create launch template.
- Choose an AMI (e.g., Amazon Linux 2).
- Select instance type.
- Configure security group.
- Add user data if needed (e.g., install web server).
- Save the launch template.

TIP: Launch templates ensure consistent instance configuration.

2 AUTO SCALING GROUP (ASG)



AUTO SCALING GROUP

- Go to EC2 > Auto Scaling Groups.
- Click Create Auto Scaling group.
- Select your launch template.
- Select your VPC and subnets.
- Choose or create a target group (will attach to ALB later).
- Configure scaling options.
- Review and create the ASG.

TIP: ASG keeps your application available during traffic changes or failures.

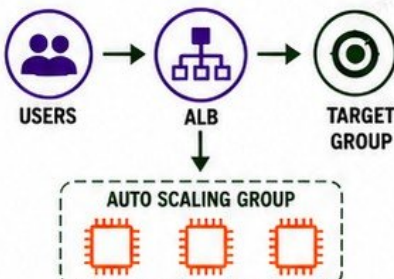
3 MINIMUM, DESIRED, AND MAXIMUM CAPACITY



- Minimum capacity: the smallest number of instances.
- Desired capacity: the ideal number of instances.
- Maximum capacity: the highest number of instances.
- ASG adjusts the number of instances between min and max.

TIP: Set desired capacity based on normal expected traffic. Set max to handle peak load.

4 CONNECT TO APPLICATION LOAD BALANCER



- Create an Application Load Balancer (ALB) and a target group.
- Attach the target group to your ASG.
- Register instances automatically with the target group.
- ALB routes traffic to healthy instances.

TIP: ALB + ASG = Highly Available and Scalable Application.

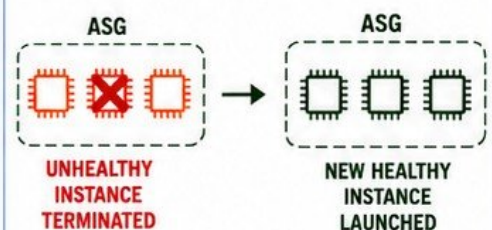
5 SIMULATE SCALING



- Generate load using a tool (e.g., Apache Bench, Locust, or k6).
- Watch CPU or request count increase.
- ASG adds more instances automatically.
- Stop load. ASG reduces instances back to desired capacity.

TIP: Scaling policies keep performance high and cost under control.

6 OBSERVE INSTANCE REPLACEMENT



- Stop or terminate an instance manually.
- ALB marks it unhealthy.
- ASG automatically terminates it.
- ASG launches a new instance to maintain desired capacity.
- Traffic continues with no downtime.

TIP: ASG automatically replaces unhealthy instances to maintain high availability.



WHAT YOU JUST DID

- ✓ Created a launch template
- ✓ Created an Auto Scaling Group
- ✓ Configured min, desired, and max capacity
- ✓ Connected ASG to ALB
- ✓ Simulated scaling under load
- ✓ Observed automatic instance replacement



NEXT STEP

Enhance your application with CloudWatch alarms, custom scaling policies, and detailed monitoring.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta

VERIQTA SOCIAL



X
veriqta



Instagram
veriqta



Telegram
veriqta

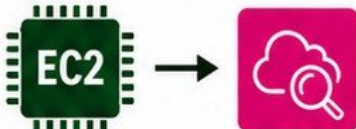


10. MONITOR AN EC2 INSTANCE WITH CLOUDWATCH



OBSERVE. ALERT. RESPOND. STAY AHEAD.

1 CLOUDWATCH METRICS

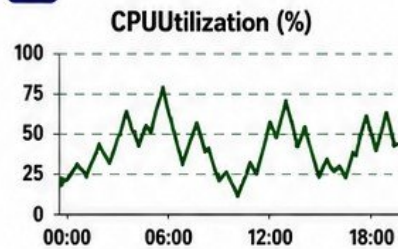


Amazon EC2 → CloudWatch

- Every EC2 instance sends metrics to Amazon CloudWatch.
- Metrics include CPU, network, disk, status checks, and more.
- CloudWatch collects, stores, and displays metrics in real time.
- No agent required for basic EC2 metrics.

TIP: CloudWatch gives you visibility into the health and performance of your instances.

2 CPU MONITORING



- Go to CloudWatch > Metrics.
- Navigate to EC2 > Per-Instance Metrics.
- Select your instance.
- Choose CPUUtilization.
- View the graph for real-time CPU usage.

TIP: Watch CPU patterns to understand normal behavior before setting alarms.

3 CREATE AN ALARM

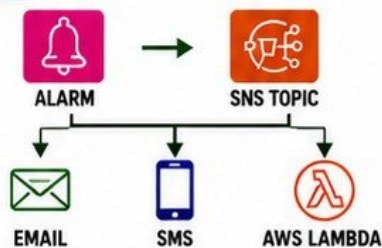


CloudWatch → ALARM

- Go to CloudWatch > Alarms.
- Click Create alarm.
- Select metric: CPUUtilization.
- Set condition (e.g., CPU > 80%).
- Choose period (e.g., 5 minutes).
- Select statistic (e.g., Average).
- Define evaluation periods (e.g., 2 out of 2).
- Name your alarm and create it.

TIP: Alarms help you detect issues before your users are affected.

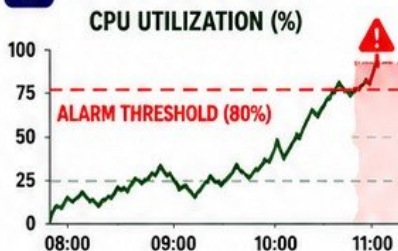
4 CONFIGURE NOTIFICATIONS



- Create an SNS topic (e.g., Email).
- Add email addresses (confirm subscriptions).
- Attach the SNS topic to your alarm.
- You will receive notifications when the alarm state changes.

TIP: Notifications ensure the right people are alerted immediately.

5 TRIGGER AN ALARM



- Generate load on the EC2 instance (e.g., run a stress command).
- CPU utilization increases.
- When CPU > 80% for 5 minutes (based on your alarm), the alarm state changes to ALARM.
- You receive a notification.

TIP: Always test alarms to confirm they work as expected.

6 INVESTIGATE THE METRIC



CloudWatch Metrics
Analyze the graph and trend.



EC2 Logs
Check system logs (e.g., /var/log).



Memory / Disk / Network
Review related metrics for root cause.



Take Action
Fix the issue, then monitor again.

TIP: Use multiple data points to find and fix the real cause.



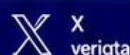
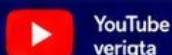
WHAT YOU JUST DID

- ✓ Viewed CloudWatch metrics
- ✓ Monitored CPU utilization
- ✓ Created a CloudWatch alarm
- ✓ Configured SNS notifications
- ✓ Triggered the alarm
- ✓ Investigated the metric



NEXT STEP

Explore advanced CloudWatch: custom metrics, dashboards, math expressions, and composite alarms.





11. CREATE AN SNS NOTIFICATION SYSTEM



EVENT-DRIVEN ALERTS. REAL-TIME NOTIFICATIONS. STAY INFORMED.

1 CREATE AN SNS TOPIC

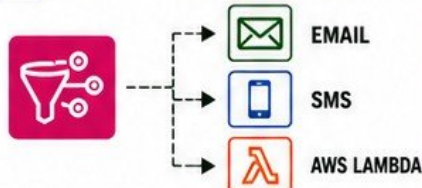


- Go to SNS in the AWS Console.
- Click "Topics" → "Create topic".
- Enter a name (e.g., "veriqta-alerts").
- Choose "Standard" topic type.
- Click "Create topic".
- Copy the Topic ARN for later use.



TIP: A topic is a communication channel for sending messages to multiple subscribers.

2 ADD A SUBSCRIPTION



- Open your topic.
- Click "Create subscription".
- Choose protocol (Email / SMS / Lambda / HTTPS).
- Enter endpoint (email, phone number, or Lambda ARN).
- Click "Create subscription".
- Confirm email subscription (if Email selected).



TIP: A subscription delivers messages from the topic to an endpoint.

3 PUBLISH MESSAGES



- Open your topic.
- Click "Publish message".
- Enter a Subject (e.g., "Test Message").
- Enter a Message.
- Click "Publish".
- The message is sent to all subscribers of the topic.



TIP: Every subscriber receives the message published to the topic.

4 CONNECT CLOUDWATCH ALARMS

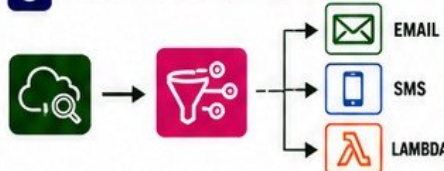


- Open CloudWatch → Alarms.
- Click "Create alarm".
- Select metric and set threshold.
- Under "Actions", click "Add action".
- Choose "Send a notification to an SNS topic".
- Select your SNS topic.
- Create the alarm.



TIP: When the alarm state changes, CloudWatch sends a notification to the SNS topic.

5 TEST NOTIFICATIONS



- Trigger the CloudWatch alarm (or publish a test message).
- Verify you receive the notification on all subscribed endpoints.
- Check spam/junk folder for emails.
- Ensure SMS phone number format includes country code.



TIP: Always test your notifications to ensure they work when you need them most.

6 UNDERSTAND EVENT-DRIVEN ALERTS



DETECT

A condition or threshold is met.



NOTIFY

SNS sends the message to subscribers.



RESPOND

Teams or systems take action immediately.

- Event-driven alerts reduce response time.
- Improve system reliability and uptime.
- Essential for DevOps and SRE practices.



TIP: Fast alerts lead to fast actions and fewer outages.



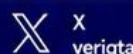
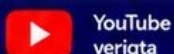
WHAT YOU JUST DID

- ✓ Created an SNS topic
- ✓ Added a subscription
- ✓ Published test messages
- ✓ Connected CloudWatch alarms
- ✓ Tested notifications
- ✓ Understood event-driven alerts



NEXT STEP

Use SNS to build real-time alerting systems for your applications and infrastructure.





12. BUILD YOUR FIRST SERVERLESS FUNCTION WITH LAMBDA

WRITE CODE. RUN ANYWHERE. SCALE AUTOMATICALLY.

1 CREATE A LAMBDA FUNCTION



AWS CONSOLE



LAMBDA FUNCTION

- Go to AWS Lambda > Functions.
- Click "Create function".
- Choose "Author from scratch".
- Enter a function name (e.g., veriqta-hello).
- Choose runtime (e.g., Python 3.12).
- Click "Create function".
- You will see a code editor with a sample code.



TIP: Lambda runs your code without provisioning or managing servers.

2 UNDERSTAND TRIGGERS

Lambda runs in response to events from AWS services.



COMMON TRIGGERS:

- S3: File upload events
- API Gateway: HTTP requests
- DynamoDB: Data changes
- SNS: Messages
- EventBridge: Scheduled or custom events



TIP: Triggers are the events that invoke your Lambda function.

3 CONFIGURE EXECUTION ROLES



EXECUTION ROLE



- Lambda needs an IAM role to run.
- Choose "Create a new role with basic Lambda permissions".
- AWS will create a role with AmazonCloudWatchLogsFullAccess.
- Click "Create function".



TIP: Execution roles give your function permissions to access AWS resources securely.

4 TEST THE FUNCTION



TEST EVENT



SUCCESS

- In the code editor, review the sample code.
- Click "Test".
- Create a new test event (or use the sample).
- Click "Test".
- You will see the output in the results panel.



TIP: Testing helps you verify your function works as expected before adding triggers.

5 VIEW CLOUDWATCH LOGS



CLOUDWATCH LOGS



LOG GROUP / STREAMS

- Every execution is logged in Amazon CloudWatch Logs.
- Go to CloudWatch > Logs.
- Click the log group named: /aws/lambda/your-function-name
- You will see log streams and log entries for each execution.



TIP: Logs help you understand what happened during function execution.

6 TROUBLESHOOT FAILURES



- Check the CloudWatch logs for error details.
- Common issues:
 - ☒ Syntax errors in code
 - ☒ Timeout (increase timeout in configuration)
 - ☒ Missing permissions in execution role
 - ☒ Incorrect environment variables
 - ☒ Invalid event or payload
- Fix the issue and test again.



TIP: Logs + errors = fast troubleshooting and reliable serverless applications.



WHAT YOU JUST DID

- ✓ Created a Lambda function
- ✓ Understood triggers
- ✓ Configured execution roles
- ✓ Tested the function
- ✓ Viewed CloudWatch logs
- ✓ Troubleshot failures



NEXT STEP

Connect your Lambda function to real AWS services like S3, API Gateway, or DynamoDB and build powerful serverless applications.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



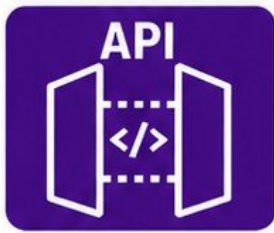
X
veriqta



Instagram
veriqta



Telegram
veriqta



13. BUILD A SERVERLESS API

BUILD. DEPLOY. INVOKE. SCALE.

AWS LAMBDA
(FUNCTION)AMAZON API
GATEWAYENDPOINT
(URL)CLIENT
(USER)CLOUDWATCH
LOGS

1 CREATE A LAMBDA FUNCTION



- GO TO AWS LAMBDA CONSOLE.
- CLICK "CREATE FUNCTION".
- CHOOSE "AUTHOR FROM SCRATCH".
- ENTER FUNCTION NAME (E.G., veriqta-api-handler).
- SELECT RUNTIME (E.G., PYTHON 3.12).
- CHOOSE OR CREATE AN EXECUTION ROLE WITH BASIC PERMISSIONS.
- CLICK "CREATE FUNCTION".



TIP: KEEP YOUR FUNCTION CODE SIMPLE AND RETURN JSON RESPONSES.

2 CONFIGURE API GATEWAY



- GO TO API GATEWAY CONSOLE.
- CLICK "CREATE API" > "HTTP API".
- CHOOSE "BUILD".
- CLICK "INTEGRATIONS" > "ADD INTEGRATION".
- SELECT "AWS LAMBDA".
- CHOOSE YOUR LAMBDA FUNCTION.
- CLICK "NEXT".



TIP: HTTP API IS SIMPLE, FAST, AND COST-EFFECTIVE FOR SERVERLESS APIS.

3 CREATE AN ENDPOINT



- CLICK "ROUTES" > "ADD ROUTE".
- SET METHOD (E.G., GET).
- SET PATH (E.G., /hello).
- SELECT YOUR LAMBDA INTEGRATION.
- CLICK "NEXT".
- CLICK "DEPLOY".
- CREATE A NEW STAGE (E.G., \$default) AND DEPLOY.
- COPY YOUR API ENDPOINT URL.



TIP: YOUR ENDPOINT URL IS NOW PUBLIC AND READY TO ACCEPT REQUESTS.

4 INVOKE LAMBDA THROUGH API



- OPEN YOUR API ENDPOINT URL IN A BROWSER OR POSTMAN.
- MAKE A GET REQUEST TO YOUR ENDPOINT.
- API GATEWAY FORWARDS THE REQUEST TO LAMBDA.
- LAMBDA PROCESSES THE REQUEST AND RETURNS A RESPONSE.



TIP: VERIFY THAT THE RESPONSE IS IN JSON FORMAT AND RETURNS THE EXPECTED DATA.

5 TEST API RESPONSES



- CHECK THE RESPONSE STATUS CODE (200 = SUCCESS).
- VALIDATE THE RESPONSE BODY.
- TEST DIFFERENT SCENARIOS (IF CONDITIONS EXIST).
- ENSURE ERROR HANDLING WORKS (TRY INVALID INPUTS).
- USE POSTMAN TO SAVE AND AUTOMATE TESTS.



TIP: GOOD APIS RETURN CLEAR STATUS CODES AND MEANINGFUL MESSAGES.

6 REVIEW LOGS



- GO TO CLOUDWATCH CONSOLE.
- OPEN "LOG GROUPS".
- FIND YOUR LAMBDA LOG GROUP (/AWS/LAMBDA/your-function-name).
- OPEN THE LOG STREAM FOR THE LATEST EXECUTION.
- REVIEW LOGS FOR INPUT, OUTPUT, AND ANY ERRORS.
- USE LOGS TO DEBUG AND IMPROVE YOUR FUNCTION.



TIP: LOGS ARE YOUR BEST FRIEND FOR DEBUGGING SERVERLESS APIS.



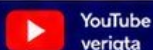
WHAT YOU JUST DID

- ✓ Created a Lambda function
- ✓ Configured API Gateway
- ✓ Created an endpoint
- ✓ Invoked Lambda via API
- ✓ Tested API responses
- ✓ Reviewed CloudWatch logs



NEXT STEP

Add authentication, request validation, CORS, custom domains, and monitoring to make your API production-ready.

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



14. BUILD A SIMPLE SERVERLESS DATABASE APPLICATION



SERVERLESS. SCALABLE. SECURE. SIMPLE.

APPLICATION ARCHITECTURE

CLIENT
(USER)

API GATEWAY

AWS LAMBDA
(FUNCTION)DYNAMODB
(TABLE)CLOUDWATCH
(LOGS)

1 CREATE A DYNAMODB TABLE



- GO TO DYNAMODB CONSOLE.
- CLICK "CREATE TABLE".
- ENTER TABLE NAME (E.G., VERIQTA-TASKS).
- CHOOSE PARTITION KEY (E.G., TASKID).
- SELECT DATA TYPE (STRING).
- LEAVE DEFAULT SETTINGS.
- CLICK "CREATE TABLE".



TIP: DYNAMODB IS A FULLY MANAGED NO-SQL DATABASE THAT SCALES AUTOMATICALLY.

2 UNDERSTAND PARTITION KEYS



TASKID	TITLE
101	Learn AWS
102	Build Project
103	Deploy App
...	...

- THE PARTITION KEY (TASKID) UNIQUELY IDENTIFIES EACH ITEM.
- ALL ITEMS WITH THE SAME TASKID ARE STORED TOGETHER.
- PARTITION KEY VALUE MUST BE UNIQUE.



TIP: CHOOSE A KEY THAT SUITS YOUR QUERY PATTERNS.

3 CREATE LAMBDA INTEGRATION



- CREATE A LAMBDA FUNCTION (PYTHON). USE BOTO3 TO ACCESS DYNAMODB.
- ADD THE DYNAMODB TABLE NAME AS AN ENVIRONMENT VARIABLE.
- LAMBDA HANDLES READ AND WRITE OPERATIONS.
- CONNECT LAMBDA TO API GATEWAY FOR ENDPOINT ACCESS.



TIP: LAMBDA IS THE BRAIN THAT PROCESSES REQUESTS AND TALKS TO DYNAMODB.

4 READ AND WRITE DATA



WRITE (CREATE ITEM)
TableName=TABLE_NAME,
Item={
 'TASKID': {'S': task_id},
 'TITLE': {'S': title},
 'STATUS': {'S': status}
}



READ (GET ITEM)
TableName=TABLE_NAME,
Key={'TASKID': {'S': task_id}}

- USE put_item() TO ADD DATA.
- USE get_item() TO RETRIEVE DATA.
- RETURN JSON RESPONSES TO THE CLIENT.



TIP: DYNAMODB API IS SIMPLE, FAST AND HIGHLY SCALABLE.

5 CONFIGURE IAM PERMISSIONS



IAM ROLE POLICY DYNAMODB

- CREATE AN IAM ROLE FOR LAMBDA.
- ATTACH THE POLICY: AmazonDynamoDBFullAccess (OR USE LEAST PRIVILEGE).
- ENSURE THE ROLE HAS PERMISSION TO PERFORM:
 - ✓ dynamodb:PutItem
 - ✓ dynamodb:GetItem
 - ✓ dynamodb:UpdateItem
 - ✓ dynamodb:Query
 - ✓ dynamodb:Scan
 - ✓ dynamodb:DeleteItem



TIP: IAM ROLES KEEP YOUR APPLICATION SECURE.

6 TEST THE COMPLETE WORKFLOW



- SEND A POST REQUEST TO CREATE A NEW ITEM.
- SEND A GET REQUEST TO RETRIEVE THE ITEM.
- VERIFY THE DATA IS STORED IN DYNAMODB.
- CHECK CLOUDWATCH LOGS (OR LAMBDA RESPONSE).
- CONFIRM THE APPLICATION WORKS END-TO-END.



TIP: YOU JUST BUILT A FULLY SERVERLESS DATABASE APPLICATION!



WHAT YOU JUST DID

- ✓ Created a DynamoDB table
- ✓ Understood partition keys
- ✓ Integrated Lambda with DynamoDB
- ✓ Performed read and write operations
- ✓ Configured IAM permissions
- ✓ Tested the complete workflow



NEXT STEP

Add more features like Update, Delete, Query, Pagination, and build a frontend using AWS Amplify or any web framework.

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



15. CREATE AN S3 FILE PROCESSING WORKFLOW



EVENT-DRIVEN. SERVERLESS. AUTOMATED.



1 UPLOAD A FILE TO S3



- GO TO S3 CONSOLE.
- CLICK "CREATE BUCKET".
- ADD A UNIQUE BUCKET NAME.
- LEAVE DEFAULT SETTINGS.
- CLICK "CREATE BUCKET".
- OPEN YOUR BUCKET.
- CLICK "UPLOAD".
- SELECT ANY FILE AND UPLOAD.



TIP: THE FILE UPLOAD WILL TRIGGER THE WORKFLOW WHEN EVENTS ARE ENABLED.

2 CONFIGURE AN S3 EVENT



- OPEN YOUR BUCKET.
- GO TO "PROPERTIES" TAB.
- SCROLL TO "EVENT NOTIFICATIONS".
- CLICK "CREATE EVENT NOTIFICATION".
- CHOOSE EVENT TYPE:
 - ✓ All object create events (s3:ObjectCreated:*)
- PREFIX: (Optional) e.g., uploads/
- SUFFIX: (Optional) e.g., .txt
- DESTINATION: Choose "Lambda".
- SELECT YOUR LAMBDA FUNCTION.
- CLICK "SAVE CHANGES".



TIP: S3 WILL INVOKE LAMBDA AUTOMATICALLY WHEN A FILE IS UPLOADED.

3 TRIGGER LAMBDA



- CREATE A LAMBDA FUNCTION (PYTHON).
- EXAMPLE CODE (PYTHON):

```
import json

def lambda_handler(event, context):
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']
        size = record['s3']['object'].get('size', 0)
        print(f"File uploaded!")
        print(f"Bucket: {bucket}")
        print(f"Key: {key}")
        print(f"Size: {size} bytes")
        return {"statusCode": 200}
```

- DEPLOY THE FUNCTION.



TIP: LAMBDA IS TRIGGERED BY S3 AND RECEIVES EVENT DATA ABOUT THE FILE.

4 PROCESS OBJECT INFORMATION



YOUR LAMBDA CAN:

- ✓ Read file metadata
- ✓ Validate file type
- ✓ Extract content
- ✓ Store results elsewhere
- ✓ Trigger further workflows

EXAMPLE: PRINTING FILE DETAILS IS THE FIRST STEP.



TIP: YOU CAN EXTEND THE FUNCTION TO PROCESS, TRANSFORM, OR MOVE THE FILE.

5 CHECK LOGS



- GO TO CLOUDWATCH CONSOLE.
- CLICK "LOG GROUPS".
- FIND YOUR LAMBDA LOG GROUP: /aws/lambda/your-function-name
- CLICK THE LATEST LOG STREAM.
- VIEW EXECUTION LOGS.
- VERIFY THE OUTPUT AND CHECK FOR ERRORS.



TIP: CLOUDWATCH LOGS SHOW EXACTLY WHAT LAMBDA DID WHEN THE EVENT FIRED.

6 TROUBLESHOOT EVENT TRIGGERS



IF LAMBDA IS NOT TRIGGERED:

- ✗ Check S3 event notification configuration
- ✗ Ensure Lambda permission is added
- ✗ Verify bucket and function are in the same region
- ✗ Check file prefix/suffix filters
- ✗ Confirm file upload after event creation
- ✗ Check CloudWatch for errors
- ✗ Re-deploy Lambda if needed



TIP: USE CLOUDWATCH AND S3 EVENT TESTING TO IDENTIFY AND FIX ISSUES.



WHAT YOU JUST DID

- ✓ Uploaded a file to S3
- ✓ Configured an S3 event
- ✓ Triggered Lambda automatically
- ✓ Processed object information
- ✓ Checked CloudWatch logs
- ✓ Troubleshoot event triggers

NEXT STEP

Enhance your workflow by:

- Adding data validation
- Storing results in DynamoDB
- Sending notifications via SNS
- Building a full serverless pipeline



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



16. CREATE A PRIVATE S3 WEBSITE WITH CLOUDFRONT



PRIVATE. SECURE. FAST. GLOBAL.



PRIVATE S3
ORIGIN



ORIGIN ACCESS
CONTROL (OAC)



CLOUDFRONT
DISTRIBUTION



USERS
(HTTPS)



SECURE CONTENT
DELIVERY

1 CREATE AN S3 ORIGIN



- GO TO S3 CONSOLE.
- CLICK "CREATE BUCKET".
- ENTER A UNIQUE BUCKET NAME (E.G., VERIQTA-PRIVATE-WEBSITE).
- UNCHECK "BLOCK ALL PUBLIC ACCESS" **ONLY IF YOU UNDERSTAND SECURITY.**
- ENABLE "BLOCK ALL PUBLIC ACCESS".
- UPLOAD YOUR WEBSITE FILES:
 - INDEX.HTML
 - ERROR.HTML (OPTIONAL)
 - /CSS, /JS, /IMAGES
- DO NOT MAKE THE BUCKET PUBLIC. KEEP IT PRIVATE.



TIP: KEEP YOUR S3 BUCKET PRIVATE. CLOUDFRONT WILL DELIVER THE CONTENT SECURELY.

2 KEEP THE BUCKET PRIVATE



- GO TO "BLOCK PUBLIC ACCESS" SETTINGS.
- CHECK ALL FOUR OPTIONS:
 - ☒ BLOCK PUBLIC ACLS
 - ☒ IGNORE PUBLIC ACLS
 - ☒ BLOCK PUBLIC BUCKET POLICIES
 - ☒ RESTRICT PUBLIC BUCKET POLICIES
- SAVE CHANGES.
- REMOVE ANY PUBLIC BUCKET POLICY IF ADDED.



TIP: THE BUCKET MUST REMAIN PRIVATE. ACCESS WILL BE CONTROLLED BY CLOUDFRONT.

3 CREATE A CLOUDFRONT DISTRIBUTION



- GO TO CLOUDFRONT CONSOLE.
- CLICK "CREATE DISTRIBUTION".
- ORIGIN DOMAIN: SELECT YOUR S3 BUCKET.
- ORIGIN ACCESS: CHOOSE "ORIGIN ACCESS CONTROL SETTINGS" (RECOMMENDED).
- ORIGIN ACCESS CONTROL: CREATE A NEW OAC (SIGV4).
- DEFAULT ROOT OBJECT: INDEX.HTML
- VIEWER PROTOCOL POLICY: REDIRECT HTTP TO HTTPS
- PRICE CLASS: SELECT APPROPRIATE.
- CLICK "CREATE DISTRIBUTION".



TIP: CLOUDFRONT WILL BE THE ONLY WAY TO ACCESS YOUR PRIVATE CONTENT.

4 CONFIGURE ORIGIN ACCESS CONTROL (OAC)



- GO TO S3 BUCKET > PERMISSIONS > BUCKET POLICY.
- ADD A POLICY THAT ALLOWS CLOUDFRONT (OAC) TO ACCESS OBJECTS.
- EXAMPLE POLICY:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket/*",
      "Condition": {
        "StringEquals": {
          "arn:aws:cloudfront::1111111111:distribution/EDFOVB06328H055"
        }
      }
    }
  ]
}
```



TIP: REPLACE DISTRIBUTION ARN WITH YOUR CLOUDFRONT DISTRIBUTION ARN.

5 TEST CACHING



- WAIT FOR THE DISTRIBUTION TO DEPLOY (STATUS: DEPLOYED).
- OPEN YOUR CLOUDFRONT DOMAIN (E.G., D123ABC.CLOUDFRONT.NET).
- REFRESH THE PAGE.
- CHECK "CACHE HIT" IN RESPONSE HEADERS. (X-CACHE: HIT FROM CLOUDFRONT)
- CLEAR CACHE (INVALIDATE) WHEN YOU UPDATE FILES.
- GO TO CLOUDFRONT > INVALIDATIONS > CREATE INVALIDATION.
- ENTER PATHS (E.G., /) AND SUBMIT.



TIP: CLOUDFRONT CACHE MAKES YOUR WEBSITE FAST GLOBALLY.

6 SECURE CONTENT DELIVERY



- FORCE HTTPS (VIEWER PROTOCOL: REDIRECT HTTP TO HTTPS).
- USE TLS CERTIFICATE (DEFAULT CLOUDFRONT CERTIFICATE OR CUSTOM ACM CERTIFICATE).
- RESTRICT ACCESS:
 - BUCKET IS PRIVATE
 - ONLY CLOUDFRONT (OAC) CAN ACCESS
- ENABLE WAF (OPTIONAL): PROTECT AGAINST COMMON ATTACKS.
- ENABLE LOGGING (OPTIONAL): SEND ACCESS LOGS TO S3.



TIP: PRIVATE S3 + CLOUDFRONT = SECURE, FAST AND GLOBAL WEBSITE DELIVERY.



WHAT YOU JUST DID

- ☒ Created a private S3 bucket
- ☒ Configured Origin Access Control (OAC)
- ☒ Created a CloudFront distribution
- ☒ Restricted S3 access to CloudFront
- ☒ Tested caching and performance
- ☒ Delivered content securely via HTTPS

NEXT STEP

Add a custom domain with Route 53, enable compression, set custom error pages, or integrate Lambda@Edge for advanced functionality.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



17. DEPLOY INFRASTRUCTURE WITH AWS CLOUDFORMATION



AUTOMATE. REPEATABLE. RELIABLE. SCALABLE.



1 INFRASTRUCTURE AS CODE BASICS



- Define your infrastructure in code.
- Version control your templates.
- Repeatable and consistent deployments.
- Track changes and maintain history.
- Reduce manual errors.



TIP: CLOUDFORMATION USES TEMPLATES (JSON OR YAML) TO PROVISION AWS RESOURCES.

2 WRITE A SIMPLE TEMPLATE

example-template.yaml

```

AWSTemplateFormatVersion: '2010-09-09'
Description: Simple CloudFormation Template

Resources:
  MyS3Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: veriqa-demo-bucket
      OwnershipControls:
        Rules:
          - ObjectOwnership: BucketOwnerPreferred

  MyEC2Instance:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: t2.micro
      ImageId: ami-0c55b159cbf1e1f0
      KeyName: my-key
      Tags:
        - Key: Name
          Value: Veriqa-Instance
  
```



TIP: YAML IS CLEANER AND EASIER TO READ FOR COMPLEX INFRASTRUCTURE.

3 CREATE AWS RESOURCES



- Go to CloudFormation Console.
- Click "Create Stack".
- Choose "With new resources (standard)".
- Upload your template file.
- Click "Next".
- Enter Stack Name (e.g., veriqa-demo-stack).
- Configure parameters (if any).
- Configure stack options (leave default).
- Acknowledge IAM permissions.
- Click "Create Stack".



TIP: CLOUDFORMATION WILL CREATE ALL RESOURCES DEFINED IN YOUR TEMPLATE.

4 UNDERSTAND STACKS



- A stack is a collection of AWS resources managed as a single unit.
- Stack status examples:
 - ✓ CREATE_IN_PROGRESS
 - ✓ CREATE_COMPLETE
 - ✓ UPDATE_IN_PROGRESS
 - ✓ UPDATE_COMPLETE
 - ✗ ROLLBACK_IN_PROGRESS
 - ✗ ROLLBACK_COMPLETE
- View events to track progress.
- Outputs return important values.



TIP: DELETE A STACK TO REMOVE ALL ASSOCIATED RESOURCES IN THE CORRECT ORDER.

5 UPDATE THE STACK



- Go to your stack.
- Click "Update".
- Choose "Replace current template".
- Upload the new template file.
- Click "Next".
- Review changes.
- Click "Update Stack".
- CloudFormation updates only what changed.
- No downtime for supported updates.



TIP: USE PARAMETERIZATION TO MAKE TEMPLATES FLEXIBLE AND REUSABLE.

6 DELETE RESOURCES SAFELY



- Go to your stack.
- Click "Delete".
- Acknowledge deletion.
- Click "Delete Stack".
- CloudFormation deletes all resources in reverse order.
- Verify status until DELETE_COMPLETE.
- Use least privilege IAM roles.
- Ensure backups if needed before deletion.



TIP: DELETING THE STACK IS THE SAFEST WAY TO CLEAN UP ALL RESOURCES.

USEFUL AWS CLI COMMANDS

```

# Create stack
aws cloudformation create-stack \
  --stack-name veriqa-demo-stack \
  --template-body file://template.yaml \
  --capabilities CAPABILITY_IAM

# List stacks
aws cloudformation list-stacks

# Update stack
aws cloudformation update-stack \
  --stack-name veriqa-demo-stack \
  --template-body file://updated-template.yaml \
  --capabilities CAPABILITY_IAM

# Delete stack
aws cloudformation delete-stack \
  --stack-name veriqa-demo-stack
  
```



BEST PRACTICES

- ✓ Use version control for templates.
- ✓ Use parameters for flexibility.
- ✓ Use outputs to share information.
- ✓ Use mappings and conditions for reusability.
- ✓ Enable termination protection for critical stacks.
- ✓ Review changes before update.
- ✓ Use least privilege IAM roles.
- ✓ Tag resources for cost tracking.
- ✓ Test templates in a non-production environment.



- ✓ Learned Infrastructure as Code
- ✓ Wrote a simple CloudFormation template
- ✓ Created AWS resources with a stack
- ✓ Understood how stacks work
- ✓ Updated the stack with changes safely
- ✓ Deleted all resources safely

WHAT YOU JUST DID



YouTube
veriqa



GitHub
veriqa



LinkedIn
veriqa



X
veriqa



Instagram
veriqa



Telegram
veriqa



18. BUILD A SIMPLE AWS BACKUP PROJECT



PROTECT. RECOVER. VERIFY.



1 CREATE AN EBS VOLUME



- Go to EC2 Console.
- Create a new EBS Volume.
- Volume Type: gp3
- Size: 8 GiB (or any size)
- Availability Zone: Choose your AZ
- Click "Create Volume".
- Note the Volume ID.

```
aws ec2 create-volume \
  --availability-zone us-east-1a \
  --size 8 \
  --volume-type gp3 \
  --tag-specifications 'ResourceType=volume,\
  Tags=[{Key=Name,Value=veriqta-backup-volume}]'
```

TIP: EBS VOLUMES ARE BLOCK STORAGE AND ARE NOT AUTOMATICALLY BACKED UP. SNAPSHOTS ARE USED FOR BACKUPS.

2 STORE TEST DATA



- Launch an EC2 instance (Amazon Linux 2).
- Attach the EBS volume to the instance.
- Initialize the volume.
- Mount the volume.
- Store some test data.

```
# Initialize and mount the volume
sudo mkfs -t ext4 /dev/xvdf
sudo mkdir /data
sudo mount /dev/xvdf /data

# Create test data
echo "VERIQTA AWS BACKUP TEST DATA" | \
sudo tee /data/test.txt
sudo df -h | grep xvdf
```

TIP: MAKE SURE THE VOLUME IS PROPERLY MOUNTED AND DATA IS WRITTEN BEFORE TAKING A SNAPSHOT.

3 CREATE A SNAPSHOT



- Go to EC2 Console.
- Select your EBS volume.
- Click "Actions" → "Create Snapshot".
- Add a description and Name.
- Click "Create Snapshot".
- Wait until the snapshot status is "Completed".

```
aws ec2 create-snapshot \
  --volume-id vol-0abcd1234efgh5678 \
  --description "VERIQTA Backup Snapshot" \
  --tag-specifications 'ResourceType=snapshot,\
  Tags=[{Key=Name,Value=veriqta-backup-snap}]'
```

TIP: SNAPSHOTS ARE INCREMENTAL AND STORE ONLY CHANGES, SAVING SPACE AND COST.

4 SIMULATE DATA LOSS



- Stop your EC2 instance.
- Detach the EBS volume.
- Delete the volume (this simulates data loss).

```
# Detach volume
aws ec2 detach-volume \
  --volume-id vol-0abcd1234efgh5678 \
  --instance-id i-0abcd1234efgh5678 \
  --device /dev/xvdf
```

```
# Delete volume
aws ec2 delete-volume \
  --volume-id vol-0abcd1234efgh5678
```

TIP: DELETING THE VOLUME DOES NOT DELETE THE SNAPSHOT. THE DATA IS STILL SAFE IN THE SNAPSHOT.

5 RESTORE FROM SNAPSHOT



- Go to EC2 Console → Snapshots.
- Select your snapshot.
- Click "Actions" → "Create Volume".
- Choose the same AZ and volume type.
- Wait until the new volume is "Available".
- Attach the new volume to your EC2 instance.
- Mount the volume.

```
# Create volume from snapshot
aws ec2 create-volume \
  --snapshot-id snap-0abcd1234efgh5678 \
  --availability-zone us-east-1a \
  --volume-type gp3 \
  --tag-specifications 'ResourceType=volume,\
  Tags=[{Key=Name,Value=veriqta-restored-volume}]'
```

```
# Attach and mount (after creating volume)
sudo mkfs -t ext4 /dev/xvdf
sudo mkdir /data
sudo mount /dev/xvdf /data
```

TIP: RESTORING FROM A SNAPSHOT CREATES A NEW VOLUME WITH ALL YOUR BACKED UP DATA.

6 VERIFY RECOVERED DATA



- Check the mounted volume.
- Verify the test file exists.
- Read the data to confirm it matches.

```
ls -l /data/
cat /data/test.txt
df -h | grep xvdf
```

SUCCESS!
YOUR DATA WAS SUCCESSFULLY RESTORED FROM THE SNAPSHOT.

TIP: REGULAR SNAPSHOTS ARE YOUR SAFETY NET. TEST RESTORES TO BE PREPARED.

>_ USEFUL AWS CLI COMMANDS

LIST SNAPSHOTS

```
aws ec2 describe-snapshots \
  --owner-ids self
```

LIST VOLUMES

```
aws ec2 describe-volumes \
  --filters "Name=tag:Name,\
  Values=veriqta-backup-volume"
```

CHECK SNAPSHOT STATUS

```
aws ec2 describe-snapshots \
  --snapshot-ids snap-0abcd1234efgh5678
```

DELETE SNAPSHOT

```
aws ec2 delete-snapshot \
  --snapshot-id snap-0abcd1234efgh5678
```



BEST PRACTICES

- ✓ Take regular snapshots of critical volumes.
- ✓ Use AWS Backup for automated backups.
- ✓ Follow the 3-2-1 backup strategy.
- ✓ Encrypt snapshots for security.
- ✓ Test restore process regularly.
- ✓ Use lifecycle policies to manage old snapshots.
- ✓ Monitor backup status with CloudWatch.



✓ Created an EBS volume

✓ Stored test data

✓ Created a snapshot (backup)

✓ Simulated data loss

✓ Restored from snapshot to a new volume

✓ Verified data was recovered



19. TROUBLESHOOT A
BROKEN AWS APPLICATION

DIAGNOSE. REPAIR. VERIFY.

EC2 WEBSITE
UNREACHABLESECURITY-GROUP
RULE ISSUEBROKEN
ROUTEFAILED ALB
HEALTH CHECKLOG
INVESTIGATIONDIAGNOSE,
REPAIR & VERIFY1 SYMPTOM: EC2 WEBSITE
UNREACHABLE

Users cannot access the website.
The browser shows:
"ERR_CONNECTION_TIMED_OUT"
or "This site can't be reached".

CHECK:

- Is the instance running?
- Is the security group correct?
- Is the route correct?
- Is the ALB healthy?



TIP: ALWAYS START WITH THE
SYMPTOM AND CHECK LAYER BY
LAYER.

2 CHECK SECURITY GROUP
(INCORRECT RULE)

- The security group does not allow
inbound traffic on port 80 (HTTP).

How to Check:

- Go to EC2 → Security Groups.
- Select the SG attached to the instance.
- Check Inbound Rules.

INCORRECT RULE (EXAMPLE)

Type	Protocol	Port Range	Source
SSH	TCP	22	0.0.0.0/0

FIX: Add inbound rule for HTTP.

CORRECT RULE

Type	Protocol	Port Range	Source
HTTP	TCP	80	0.0.0.0/0



TIP: SECURITY GROUPS ARE
STATEFUL. RETURN TRAFFIC IS
ALLOWED AUTOMATICALLY.

3 CHECK ROUTE TABLE
(BROKEN ROUTE)

- The route table may be missing the route
to the Internet Gateway (IGW).

How to Check:

- Go to VPC → Route Tables.
- Select the route table associated with
the subnet.
- Check if 0.0.0.0/0 points to IGW.

BROKEN ROUTE (EXAMPLE)

Destination	Target
10.0.0.0/16	local

FIX: Add route to IGW.

CORRECT ROUTE

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-0abcd1234567890



TIP: PUBLIC SUBNETS NEED A ROUTE
TO THE INTERNET GATEWAY.

4 CHECK ALB HEALTH CHECK
(FAILED HEALTH CHECK)

- The target is unhealthy because the
health check is failing.

How to Check:

- Go to EC2 → Load Balancers.
- Select your ALB.
- Go to "Target Groups" → Targets.
- Check the Health Status.

COMMON ISSUES

- Wrong health check path (e.g., /health)
- Application not responding
- Security group blocking ALB → EC2 traffic

FIX:

- Update the health check path.
- Ensure the application is running.
- Allow inbound traffic from ALB SG to
instance on the correct port.



TIP: HEALTH CHECK PATH MUST RETURN
HTTP 200 OK FOR SUCCESS.

5 INVESTIGATE APPLICATION LOGS



- Application logs help identify the real issue.

Where to Check:

- EC2: /var/log/nginx/access.log
- EC2: /var/log/nginx/error.log
- Application logs (e.g., /var/log/app.log)

Example: Check Nginx Error Log

sudo tail -f /var/log/nginx/error.log

Common Errors:

- 502 Bad Gateway → App not running
- 403 Forbidden → Permission issue
- 404 Not Found → Wrong path
- 500 Internal Server Error → App bug



TIP: LOGS TELL YOU WHAT THE USER
DOES NOT SEE.

6 DIAGNOSE, REPAIR & VERIFY

Final Checklist

- ✓ Security group allows HTTP (80)
- ✓ Route table has 0.0.0.0/0 → IGW
- ✓ ALB health check is passing
- ✓ Application is running
- ✓ Logs show no critical errors

Verify

- Open the website in a browser.
- Confirm HTTP 200 OK.
- Monitor in CloudWatch.



SUCCESS!
THE APPLICATION IS NOW
ACCESSIBLE AND WORKING.

USEFUL AWS CLI COMMANDS

CHECK INSTANCE STATUS

```
aws ec2 describe-instances \
  --instance-ids i-1234567890 \
  --query "Reservations[].Instances[]
.State.Name" --output text
```

CHECK SECURITY GROUPS

```
aws ec2 describe-security-groups \
  --group-ids sg-1234567890
```

CHECK ROUTE TABLES

```
aws ec2 describe-route-tables \
  --route-table-ids rtb-1234567890
```

CHECK ALB TARGET HEALTH

```
aws elbv2 describe-target-health \
  --target-group-arn
arn:aws:elasticloadbalancing:region:acct:
targetgroup/my-tg/1234567890abcdef
```

CHECK LOGS (CLOUDWATCH)

```
aws logs describe-log-streams \
  --log-group-name /aws/ec2/my-app
```

VIEW LOG EVENTS

```
aws logs get-log-events \
  --log-group-name /aws/ec2/my-app \
  --log-stream-name stream-name
```

BEST PRACTICES

- ✓ Follow a systematic troubleshooting approach.
- ✓ Check one layer at a time.
- ✓ Use CloudWatch for monitoring and logs.
- ✓ Keep security groups minimal but functional.
- ✓ Document issues and resolutions.



✓ Identified the
symptom

✓ Checked and
fixed SG rule

✓ Fixed the
route issue

✓ Fixed ALB health
check

✓ Investigated
application logs

✓ Verified the
application is working

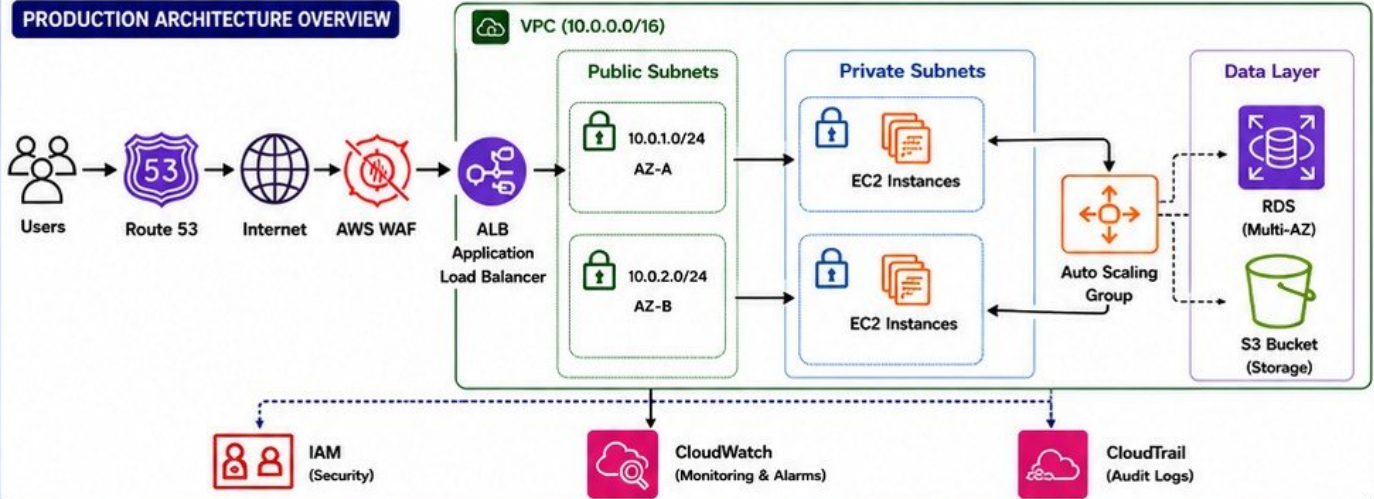
YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



FINAL PROJECT: BUILD A BEGINNER PRODUCTION-STYLE AWS ARCHITECTURE

SCALABLE. SECURE. HIGHLY AVAILABLE. PRODUCTION-READY.

PRODUCTION ARCHITECTURE OVERVIEW



1 CREATE A CUSTOM VPC

- Create VPC: 10.0.0.0/16
- Create Internet Gateway
- Create Public Subnets (2 AZs)
- Create Private Subnets (2 AZs)
- Create Route Tables
- Associate subnets with route tables
- Enable DNS hostnames

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-xxxx
aws ec2 create-subnet --vpc-id vpc-xxxx \
  --cidr-block 10.0.1.0/24 --availability-zone us-east-1a
```

2 DEPLOY APPLICATION LOAD BALANCER

- Create ALB in public subnets
- Enable HTTP (80) and HTTPS (443) listeners
- Attach security group (allow 80,443)

```
aws elbv2 create-load-balancer \
  --name app-alb \
  --subnets subnet-public1 subnet-public2 \
  --security-groups sg-alb \
  --type application \
  --scheme internet-facing
```

3 LAUNCH EC2 APPLICATION TIER

- Launch EC2 instance (Amazon Linux 2)
- Install and start Apache/Nginx
- Create Launch Template
- Attach IAM Role (EC2 Role)

```
aws ec2 run-instances --image-id ami-0e55b159cbf1ef0 \
  --instance-type t3.micro \
  --subnet-id subnet-private1 \
  --security-group-ids sg-ec2 \
  --iam-instance-profile Name=EC2Role
```

4 CONFIGURE AUTO SCALING GROUP

- Create Auto Scaling Group (private subnets)
- Attach Launch Template
- Set desired capacity (2)
- Set min (1) and max (4)
- Attach to ALB Target Group
- Enable scaling policies (CPU > 60%)

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name app-asg \
  --launch-template LaunchTemplateName=app-lt,Version=1 \
  --min-size 1 --max-size 4 --desired-capacity 2 \
  --vpc-zone-identifier subnet-private1,subnet-private2 \
  --target-group-arns arn:aws:elasticloadbalancing::...
```

5 SET UP RDS DATABASE (MULTI-AZ)

- Create RDS (MySQL/Aurora)
- Deploy in private subnets
- Enable Multi-AZ for high availability
- Store endpoint for application

```
aws rds create-db-instance \
  --db-instance-identifier appdb \
  --engine mysql \
  --allocated-storage 20 \
  --db-instance-class db.t3.micro \
  --multi-az \
  --db-subnet-group-name rds-subnet-group \
  --vpc-security-group-ids sg-rds \
  --master-username admin --master-user-password YourPass123
```

6 CONFIGURE S3 STORAGE

- Create S3 bucket for uploads/static files
- Enable versioning
- Block public access
- (Optional) Enable lifecycle policies

```
aws s3 mb s3://veriqta-app-storage
aws s3api put-bucket-versioning --bucket veriqta-app-storage \
  --versioning-configuration Status=Enabled
aws s3api put-public-access-block --bucket veriqta-app-storage \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true, \
    BlockPublicPolicy=true,RestrictPublicBuckets=true
```

7 ENABLE CLOUDWATCH MONITORING

- Enable detailed monitoring
- Create CloudWatch Alarms (CPU > 60%)
- Monitor ALB, EC2, RDS, ASG
- Set alarm notifications (SNS)

```
aws cloudwatch put-metric-alarm \
  --alarm-name CPU-High \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average --period 300 \
  --threshold 60 --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 --alarm-actions arn:aws:sns::...
```

8 CONFIGURE IAM SECURITY

- Create IAM roles for EC2 and application
- Principle of least privilege
- Enable MFA for users
- Use IAM policies for fine-grained access

```
aws iam create-role --role-name EC2Role \
  --assume-role-policy-document file://ec2-trust.json
aws iam put-role-policy --role-name EC2Role \
  --policy-name S3ReadAccess --policy-document file://s3-read.json
```

9 TESTING & FAILURE SIMULATION

- Test application via ALB DNS
- Simulate EC2 instance failure (terminate)
- Verify Auto Scaling replaces instance
- Simulate AZ failure (disable AZ)
- Verify Multi-AZ RDS failover
- Test S3 file upload/download

10 FINAL PRODUCTION CHECKLIST

- | | | |
|-------------------------------------|----------------------------------|------------------------------|
| ✓ VPC with public & private subnets | ✓ S3 bucket secured | ✓ Architecture tested |
| ✓ Internet Gateway & Route Tables | ✓ CloudWatch monitoring & alarms | ✓ Failure scenarios tested |
| ✓ ALB with HTTP/HTTPS listeners | ✓ IAM roles with least privilege | ✓ Logs and metrics verified |
| ✓ EC2 in private subnets | ✓ Security groups configured | ✓ Cost optimization reviewed |
| ✓ Auto Scaling Group configured | ✓ WAF enabled (optional) | ✓ Documentation completed |
| ✓ RDS Multi-AZ enabled | ✓ Backups enabled for RDS | |

PROJECT OUTCOME

- ✓ You have successfully built a production-style AWS architecture that is scalable, secure, highly available, and monitored.

This architecture is ready for real-world applications and future growth.

VERIQTA | BUILD • AUTOMATE • SCALE • MONITOR



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

VERIQTA

VERIQTA

VERIQTA

VERIQTA

VERIQTA